Design, Testing and Version Control

Course notes - Department of Computer Science

1. Requirements

Functional requirements say what the system must do; non-functional requirements constrain how well - latency, availability, security, accessibility. Non-functional requirements are the ones most often discovered too late, after the architecture has already foreclosed them.

A requirement that cannot be tested cannot be verified. "The system should be fast" is not a requirement; "95th percentile response under 200ms at 1000 concurrent users" is.

2. Coupling and cohesion

Cohesion measures how strongly the responsibilities within a module belong together; coupling measures how much modules depend on one another. Good design maximises cohesion and minimises coupling, because that is what lets one part change without disturbing the rest.

Depending on an interface rather than a concrete implementation reduces coupling and is the basis of dependency inversion.

3. Design patterns

Patterns are named solutions to recurring design problems. Creational patterns such as factory and builder decouple construction from use. Structural patterns such as adapter and decorator compose objects into larger structures. Behavioural patterns such as observer and strategy assign responsibility for algorithms and communication.

Patterns are vocabulary, not goals. Applying one where the problem does not exist adds indirection and costs clarity.

4. Testing strategy

Unit tests exercise a single module in isolation and should be fast and numerous. Integration tests verify that modules work together, particularly across process and network boundaries. End-to-end tests drive the whole system as a user would; they catch the most and cost the most to maintain.

The pyramid recommends many unit tests, fewer integration tests and fewer still end-to-end tests. Coverage measures which lines ran, not whether behaviour is correct, so treat it as a diagnostic rather than a target.

5. Version control

A commit records a snapshot together with its parents, forming a directed acyclic graph. Branches are movable pointers into that graph, which is why creating one is cheap.

Merging combines divergent histories and records both parents; rebasing replays commits onto a new base, producing a linear history at the cost of rewriting commit

identities. Never rebase history that others have already pulled.